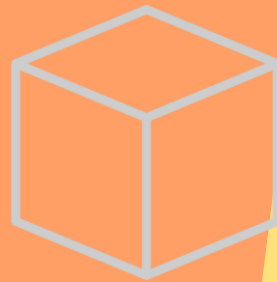


Acceso a Variables y Funciones Especiales



Scope de variables en Python: local, global y nonlocal

El Modelo LEGB

- L** **Local:** Variables definidas dentro de una función.
- E** **Enclosing:** Variables en funciones envolventes (clásico en *closures*).
- G** **Global:** Variables definidas a nivel superior del módulo o script.
- B** **Built-ins:** Nombres predefinidos en Python (ej. `print`, `len`).

Modificadores de Scope

global : Permite reasignar una variable del entorno global desde dentro de una función local.

nonlocal : Permite modificar variables en el scope *enclosing* más cercano (sin llegar al global).

Buenas Prácticas

Evite el uso excesivo de `global`. En su lugar, prefiera pasar datos como parámetros, retornar resultados o utilizar programación orientada a objetos.

Ejemplo 1: Uso de nonlocal en closures

```
x = 1 # Global scope
def outer():
    x = 10 # Enclosing scope
    def inner():
        nonlocal x
        x += 1
    print("inner:", x) # Imprime 11
    inner()
    print("outer:", x) # Imprime 11
outer()
print("global x:", x) # Imprime 1
```

Ejemplo 2: Uso de global

```
count = 0
def incrementar():
    global count
    count += 1
incrementar()
print("Count:", count) # Imprime 1
```

</> ¿Qué son las variables internas en Python?

Variables Internas Definidas por el Intérprete

Python proporciona variables y atributos especiales que son definidos por el intérprete o por convenios establecidos. Estas variables internas permiten la reflexión, depuración, serialización y metaprogramación.

- ✓ `__name__`: Nombre del módulo o clase
- ✓ `__doc__`: Documentación de la clase
- ✓ `__dict__`: Diccionario de atributos
- ✓ `__class__`: Clase del objeto

 **Uso principal:** Reflexión, depuración, serialización y metaprogramación

Ejemplos de Uso

```
__name__  
'MiClase'
```

```
__doc__  
'Demo class'
```

```
__module__  
'__main__'
```

Ejemplo de Variables Internas

```
class DemoClass:  
    """Clase de demostración"""  
  
    def __init__(self, valor):  
        self.valor = valor  
  
# Crear instancia  
obj = DemoClass(42)  
  
# Acceder a variables internas  
  
print(DemoClass.__name__)      # 'DemoClass'  
print(DemoClass.__doc__)      # 'Clase de demostración'  
print(obj.__class__)           # <class '__main__.DemoClass'>  
print(obj.__dict__)            # {'valor': 42}  
print(DemoClass.__module__)    # '__main__'  
print(DemoClass.__qualname__)  # 'DemoClass'
```

Introspección de Objetos

Las variables internas permiten inspeccionar objetos en tiempo de ejecución, lo que es fundamental para el debugging y la metaprogramación en frameworks de IA.



Convenciones de nomenclatura: _ y __



Protegido (_)

Convención de "protegido"

Características

El prefijo _ indica que el atributo es "protegido" por convención. No es privado realmente, pero señala que no debe usarse fuera de la clase.

- ✓ Accesible desde fuera de la clase
- ✓ Ignorado por `from module import *`
- ✓ Convención, no restricción técnica

```
class Ejemplo:
    def __init__(self):
        self.publico = 1
        self._protegido = 2
obj = Ejemplo()
print(obj._protegido)  # Acceso permitido
```



Privado (__)

Name Mangling

Características

El prefijo __ activa el "name mangling" para evitar colisiones de nombres en herencia. Se reescribe como `_Clase__atributo`.

- ✓ Acceso restringido (no es seguro)
- ✓ Evita colisiones en herencia
- ✓ Accesible vía `_Clase__atributo`

```
class Ejemplo:
    def __init__(self):
        self.__privado = 3
obj = Ejemplo()
print(obj._Ejemplo__privado)  # Acceso con mangling
```

Reflexión: ¿Por qué convenciones y no privacidad estricta?



Filosofía "Consent of Adults" de Python

Python sigue la filosofía de que "los adultos deben ser tratados como adultos". En lugar de restricciones técnicas rígidas, Python usa convenciones de nomenclatura para comunicar la intención del programador.



Ventajas

¿Qué ventajas aporta esta filosofía en términos de:

- ✓ Testeo y depuración
- ✓ Extensibilidad
- ✓ Flexibilidad



Riesgos

¿Cuándo podría ser un riesgo:

- ✓ Acceso accidental
- ✓ Modificación incorrecta
- ✓ Documentación



Balance

¿Cómo encontrar el equilibrio entre:

- ✓ Libertad vs Control
- ✓ Flexibilidad vs Seguridad



Dinámica de Participación

 10 minutos

3 minutos: Pensar individualmente sobre las preguntas | 4 minutos: Compartir en parejas | 3 minutos: Plenario para discutir conclusiones

</> Name Mangling: Cómo funciona __atributo

🔧 ¿Qué es el Name Mangling?

El name mangling es el proceso mediante el cual Python reescribe los atributos que comienzan con __ (doble guion bajo) para evitar colisiones en la herencia.

El atributo __x se transforma en _Clase__x.



🔍 **Objetivo:** Evitar colisiones de nombres en jerarquías de herencia complejas

🛡️ Protección vs Accesibilidad

Aunque el atributo se "privatiza", sigue siendo accesible desde fuera de la clase:

__x → _Clase__x → Accesible

⚠️ **Nota:** No es seguridad real, solo una convención de nomenclatura

> Ejemplo de Name Mangling

```
class MiClase:
    def __init__(self):
        self.__privado = 42 # Se convierte en _MiClase__privado
        self._protegido = 10 # Solo convención
# Crear instancia
obj = MiClase()
# Intentar acceder directamente
print(hasattr(obj, "__privado")) # False
# Acceder con name mangling
print(obj._MiClase__privado) # 42
# Ver atributos disponibles
print(dir(obj)) # ['_MiClase__privado', ...]
```

✓ Verificación de Acceso

hasattr(obj, "__privado") → False

hasattr(obj, "_MiClase__privado") → True

Protegido vs Privado (por convención)



Protegido (_x)

Convención de uso

Características principales

El prefijo _ indica que el atributo es "protegido" por convención. Es accesible desde fuera pero señala que no debe usarse fuera de la clase.

- ✓ Accesible desde fuera de la clase
- ✓ Ignorado por `from module import *`
- ✓ Convención, no restricción técnica

```
class Ejemplo:
    def __init__(self):
        self.publico = 1
        self._protegido = 2
obj = Ejemplo()
print(obj._protegido) # Acceso permitido
```

 **Uso recomendado:** Para atributos internos que pueden cambiar



Privado (__x)

Name Mangling

Características principales

El prefijo __ activa el "name mangling" para evitar colisiones de nombres en herencia. Se reescribe como `_Clase__atributo`.

- ✓ Acceso restringido (no es seguro)
- ✓ Evita colisiones en herencia
- ✓ Accesible vía `_Clase__atributo`

```
class Ejemplo:
    def __init__(self):
        self.__privado = 3
obj = Ejemplo()
print(obj._Ejemplo__privado) # Acceso con mangling
```

 **Uso recomendado:** Para evitar colisiones en herencia compleja

Métodos Mágicos (Dunder) en Python

🔧 ¿Qué son los Dunder Methods?

Dunder viene de "Double UNDERscore". Son métodos con nombres como `__init__` o `__str__` que permiten personalizar el comportamiento de los objetos frente a operadores y funciones integradas de Python.



Representación de objetos

`__repr__`: Representación oficial (para desarrolladores).

`__str__`: Representación informal (usada por `print`).



Sobrecarga de operadores

`__add__`: Define el comportamiento del operador suma (+).

`__eq__`: Define el comportamiento de igualdad (==).



Protocolos y utilidades

`__len__`: Retorna la longitud (usado por `len()`).

💡 Regla de Oro

Al sobrecargar operadores binarios (como `__add__`), verifica el tipo del otro objeto. Si no sabes cómo operar con él, devuelve `NotImplemented` para permitir que Python intente operaciones inversas.

Ejemplo Práctico: Clase Vector con Dunder Methods

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        # Para desarrolladores e interactivo
        return f"Vector(x={self.x}, y={self.y})"
    def __str__(self):
        # Para usuarios (ej: print)
        return f"({self.x}, {self.y})"
    def __add__(self, other):
        if not isinstance(other, Vector):
            return NotImplemented
        return Vector(self.x + other.x, self.y + other.y)

    def __len__(self):
        return 2 # Un vector 2D siempre tiene 2 elementos

# Uso práctico
v1 = Vector(2, 3)
v2 = Vector(1, -1)

print(v1 + v2)      # Muestra: (3, 2)
print(repr(v1))     # Muestra: Vector(x=2, y=3)
print(len(v1))      # Muestra: 2
```


✏ Métodos mágicos: `__init__`, `__str__`, `__repr__`

i Métodos Mágicos de Representación

Los métodos mágicos (dunder methods) permiten integrar tus clases con el ecosistema Python. Los tres más importantes para la representación de objetos son:

📦 `__init__`, `__str__`, `__repr__`

`__init__`: Inicializa el estado del objeto

`__str__`: Representación legible para humanos

`__repr__`: Representación no ambigua para desarrolladores

💡 Regla de oro: `__repr__` debe ser inequívoco; `__str__` debe ser legible

</> Diferencias Clave

`__str__`

Para usuarios finales
Legible, amigable

`__repr__`

Para desarrolladores
Preciso, completo

> Ejemplo: Clase Punto

```
class Punto:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __str__(self):
        return f"({self.x}, {self.y})"
    def __repr__(self):
        return f"Punto(x={self.x}, y={self.y})"

# Uso
p = Punto(3, 4)
print(str(p))    # (3, 4)
print(repr(p))   # Punto(x=3, y=4)
print(p)         # (3, 4)
```

💡 Consejos de Implementación

- ✓ Implementa `__repr__` primero para debugging
- ✓ `__str__` debe ser conciso y legible
- ✓ Incluye toda la información necesaria en `__repr__`

Métodos de comparación: `__eq__`, `__lt__`, `__gt__`

Implementación de métodos de comparación

Python 3.x OOP

```
1 class Punto:
2     """Clase que representa un punto en 2D con métodos de comparación"""
3     def __init__(self, x, y):
4         self.x = x
5         self.y = y
6
7     def __eq__(self, otro):
8         # Comparación de igualdad (==)
9         if not isinstance(otro, Punto):
10            return NotImplemented
11        return (self.x, self.y) == (otro.x, otro.y)
12
13    def __lt__(self, otro):
14        # Comparación menor que (<)
15        if not isinstance(otro, Punto):
16            return NotImplemented
```

Notas importantes

- ✓ Usa `functools.total_ordering` para derivar comparaciones restantes
- ✓ Debe ser consistente con el invariante del objeto
- ✓ Retorna `NotImplemented` para tipos incompatibles

Mejores prácticas

- ✓ Implementa `__repr__` para debugging
- ✓ Considera tolerancia para floats
- ✓ Documenta el orden de comparación

Métodos aritméticos: `__add__`, `__sub__`, `__mul__`

Implementación de operaciones aritméticas en clase Vec

Python 3.x OOP

```
1 class Vec:
2     """Clase vector 2D con operaciones aritméticas"""
3     def __init__(self, x, y):
4         self.x = x
5         self.y = y
6
7     def __add__(self, otro):
8         # Suma de vectores (+)
9         if not isinstance(otro, Vec):
10            return NotImplemented
11            return Vec(self.x + otro.x, self.y + otro.y)
12
13    def __sub__(self, otro):
14        # Resta de vectores (-)
15        if not isinstance(otro, Vec):
16            return NotImplemented
```

Métodos adicionales recomendados

`__radd__`

Suma inversa (a + vec)

`__iadd__`

Suma in-place (+=)

`__rmul__`

Mult. inversa (3 * vec)

Consideraciones importantes

- ✓ Implementa `__radd__` para operandos invertidos
- ✓ Usa `__iadd__` para operaciones mutables (+=)
- ✓ Valida tipos en todos los métodos

</> Ejercicio práctico: implementa __eq__



Clase Vector3D con tolerancia float

Implementa métodos mágicos para comparación de vectores 3D

🕒 5-7 min

📖 Nivel: Intermedio



Tarea principal

Implementa la clase Vector3D con los siguientes requisitos:

- ✓ `__init__`: Inicializa x, y, z
- ✓ `__eq__`: Comparación con tolerancia para floats
- ✓ `__repr__`: Representación informativa
- ✓ Pruebas: 2-3 asserts por método

```
import math
class Vector3D:
    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z
```



Pistas y consejos

- ✓ Usa `math.isclose()` para comparación de floats
- ✓ Considera tolerancia relativa y absoluta
- ✓ Implementa `__repr__` para debugging
- ✓ Prueba con valores negativos

💡 Tip: Para comparación de vectores, compara cada componente con tolerancia.

🕒 Context Managers: `__enter__` y `__exit__`

📄 ¿Qué son los Context Managers?

Los context managers implementan el protocolo `__enter__` y `__exit__` para gestionar recursos de manera segura. Permiten el uso de la sentencia `with` para garantizar la liberación de recursos.

- ✓ `__enter__`: Se ejecuta al entrar al contexto (`with`)
- ✓ `__exit__`: Se ejecuta al salir del contexto
- ✓ Excepciones: Manejo de errores automático

💡 **Ventaja:** Garantiza la liberación de recursos incluso si ocurren errores

💡 Casos de Uso Comunes

Archivos
with open()



DB
Conexiones



Locks
Threading

> Ejemplo: Clase Temporizador

```
import time

class temporizador:
    """Context manager para medir tiempo de ejecución"""
    def __enter__(self):
        self.start_time = time.perf_counter()
        print("🌀 Iniciando temporizador...")
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed = time.perf_counter() - self.start_time
        print(f"✅ Tiempo: {elapsed:.4f} segundos")
        return False # Propaga excepciones

# Uso del context manager
with temporizador():
    # Código a medir
    time.sleep(1)
```

</> Ejemplo de Uso

El context manager `temporizador` mide el tiempo de ejecución de cualquier bloque de código, garantizando que siempre se muestre el resultado.

Iteradores: `__iter__` y `__next__`

Protocolo de Iteración en Python

Los iteradores implementan el protocolo de iteración mediante los métodos `__iter__` y `__next__`. Esto permite crear objetos que pueden ser recorridos en bucles `for`.

- ✓ `__iter__()`: Devuelve el objeto iterador (self)
- ✓ `__next__()`: Devuelve el siguiente elemento o lanza `StopIteration`
- ✓ `StopIteration`: Señal de fin de iteración

→ Flujo del Protocolo

1. Se llama `iter(obj)` → `__iter__()`
2. Se llama `next(iterator)` → `__next__()`
3. Se repite hasta `StopIteration`

Características Clave

- ✓ El iterador mantiene su propio estado
- ✓ Puede usarse en múltiples bucles
- ✓ Compatible con `for`, `list()`, `tuple()`

Ejemplo: Clase Contador

```
class Contador:
    def __init__(self, n):
        self.n = n
        self.i = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.i >= self.n:
            raise StopIteration
        self.i += 1
        return self.i

# Uso del iterador
contador = Contador(5)
for num in contador:
    print(num) # 1, 2, 3, 4, 5

# Conversión a lista
lista = list(Contador(3))
print(lista) # [1, 2, 3]
```

 **Tip:** Implementa `__iter__` para hacer tu clase iterable y `__next__` para controlar la secuencia de valores.

∞ Iterador personalizado: Fibonacci

</> Implementación de iterador Fibonacci

Python 3.x **Iterator**

```
1 class Fib:
2     """Iterador personalizado para la secuencia de Fibonacci"""
3     def __init__(self, n):
4         self.n = n # Número de términos a generar
5         self.a, self.b = 0, 1 # Valores iniciales
6         self.count = 0
7
8     def __iter__(self):
9         # Reinicia el iterador
10        self.a, self.b = 0, 1
11        self.count = 0
12        return self
13
14    def next (self):
```

Concepto clave

- ✓ `__iter__` reinicia el estado y retorna self
- ✓ `__next__` genera el siguiente valor
- ✓ StopIteration termina la iteración

0 → 1 → 1 → 2 → 3 → 5 → 8

Uso práctico

- ✓ Generación lazy de números
- ✓ Memoria eficiente
- ✓ Reutilizable con `__iter__`

</> Descriptores: __get__, __set__, __delete__

¿Qué son los Descriptores?

Los descriptores son objetos que definen cómo acceder a los atributos de una clase. Implementan los métodos `__get__`, `__set__` y/o `__delete__` para controlar el acceso a los atributos a nivel de clase.

- ✓ `__get__(self, obj, type)`: Obtiene el valor del atributo
- ✓ `__set__(self, obj, value)`: Establece el valor del atributo
- ✓ `__delete__(self, obj)`: Elimina el atributo
- ✓ `__set_name__(self, owner, name)`: Se llama al crear el descriptor

💡 Uso: Validación de datos, caché, acceso controlado, propiedades computadas

> Ejemplo: Validador de Rango

```
class Rango:
    def __init__(self, min_val, max_val):
        self.min_val = min_val
        self.max_val = max_val
    def __set_name__(self, owner, name):
        self.name = name
        self.private_name = f"_{name}"
    def __get__(self, obj, objtype=None):
        if obj is None: return self
```

💡 Tipos de Descriptores

Data Descriptor

`__get__` + `__set__`

Non-data Descriptor

Solo `__get__`

🔍 Uso en IA/ML

Los descriptores son útiles para validar hiperparámetros de modelos, controlar el acceso a datos sensibles y implementar lazy loading de datasets.

__call__: Objetos Invocables

i ¿Qué es __call__?

El método especial `__call__` permite que una instancia de una clase sea invocada como si fuera una función. Esto hace que los objetos se comporten como funciones, manteniendo su estado interno.

- ✓ Sintaxis: `objeto()` equivale a `objeto.__call__()`
- ✓ Uso: Transformaciones, normalización, métricas
- ✓ Ventaja: Mantiene estado entre llamadas

💡 Caso de uso: Normalización de datos, preprocesamiento en ML/IA

💡 Ventajas de __call__

Aplicaciones comunes:

- | | |
|--------------------------------|-------------------------|
| 📊 Transformaciones matemáticas | 🧠 Capas de red neuronal |
| 📈 Métricas de evaluación | 🔧 Preprocesamiento |

</> Ejemplo: Clase Normalizar

```
class Normalizar:
    """Clase para normalizar datos"""
    def __init__(self, mu, sigma):
        self.mu = mu
        self.sigma = sigma
    def __call__(self, x):
        return (x - self.mu) / self.sigma

# Crear instancia
normalizador = Normalizar(0, 1)

# Usar como función
resultado = normalizador(3.14) # 3.14
print(resultado) # 3.14

# También se puede llamar explícitamente
resultado = normalizador.__call__(3.14)
```

✓ Sintaxis Alternativa

Llamada directa
`obj(x)`

Método explícito
`obj.__call__(x)`

Reflexión: ¿Dónde usarías `__call__` en IA/ML?



Aplicaciones de `__call__` en Inteligencia Artificial

El método `__call__` permite que las instancias de clase se comporten como funciones, lo que es fundamental en ML para crear capas, transformaciones y pipelines componibles.



Capas

Transformaciones y activaciones

- ✓ Normalización
- ✓ Activaciones
- ✓ Dropout



Métricas

Funciones de pérdida

- ✓ MSE, MAE
- ✓ Cross-entropy
- ✓ Custom metrics



Pipelines

Preprocesamiento

- ✓ Data cleaning
- ✓ Feature engineering
- ✓ Composición



Modelos

Invocación de modelos

- ✓ Inference
- ✓ Batch processing
- ✓ Ensemble

Caso de estudio: Diseña una clase de preprocesamiento

 5 min

Ejercicio: Crea una clase `Preprocesador` que use `__call__` para aplicar una serie de transformaciones (normalización, scaling, encoding) a datos de entrada. Considera: ¿Cómo harías la clase componible y reutilizable?

Comparativa de Métodos Especiales por Categorías



Construcción

<code>__new__</code>	Crear instancia	clase
<code>__init__</code>	Inicializar	inst
<code>__del__</code>	Destruir	del



Representación

<code>__repr__</code>	No ambigua	dev
<code>__str__</code>	Legible	user
<code>__format__</code>	Formato	fmt



Colecciones

<code>__len__</code>	Longitud	len()
<code>__getitem__</code>	Acceso []	[]
<code>__setitem__</code>	Asignar []	[]=



Comparación

<code>__eq__</code>	Igualdad	=
<code>__lt__</code>	Menor	<
<code>__hash__</code>	Hash	0



Aritmética

<code>__add__</code>	Suma	+
<code>__sub__</code>	Resta	-
<code>__mul__</code>	Multiplica	*



Contexto

<code>__enter__</code>	Entrar	with
<code>__exit__</code>	Salir	with
<code>__aenter__</code>	Async	async



Invocación

<code>__call__</code>	Llamar	()
<code>__get__</code>	Obtener	desc
<code>__set__</code>	Asignar	desc

🔍 Introspección: dir, vars, getattr, setattr

📄 Funciones de Introspección

Python proporciona funciones integradas para examinar objetos en tiempo de ejecución. Estas herramientas son esenciales para debugging, metaprogramación y desarrollo de frameworks.

`dir(obj)`

Lista de atributos y métodos

`vars(obj)`

Diccionario de atributos

`getattr(obj, attr)`

Obtiene valor de atributo

`setattr(obj, attr, val)`

Establece valor de atributo

💡 **Uso:** Debugging, serialización, validación de atributos

💡 Ventajas de Introspección

- ✓ Descubrimiento dinámico de APIs
- ✓ Validación de tipos en runtime
- ✓ Generación automática de documentación

> Ejemplo de Introspección

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

# Crear instancia
p = Persona("Ana", 25)

# dir() - listar atributos y métodos
print(dir(p))  # ['__class__', '__dict__', ...]

# vars() - diccionario de atributos
print(vars(p))  # {'nombre': 'Ana', 'edad': 25}

# getattr() - obtener valor
nombre = getattr(p, 'nombre')  # 'Ana'

# setattr() - establecer valor
setattr(p, 'edad', 26)
```

🔍 Comprobación de Atributos

`hasattr(obj, 'attr')` verifica si existe un atributo, útil para validación antes de acceder a atributos opcionales.

🔍 Inspección en tiempo de ejecución

🏗 Flujo de proceso de introspección

Python 3.x Introspección



👁 Inspección básica

Usa `dir()` para listar atributos, `vars()` para el diccionario de atributos, y `getattr()` para acceder dinámicamente.

```
print(dir(obj)) # Lista de atributos
print(vars(obj)) # Dict de atributos
```

⚙ Metaprogramación

Usa `inspect` para análisis avanzado, `__dict__` para estado, y `__class__` para tipo.

```
import inspect
inspect.getmembers(obj) # Detalles completos
```


⚙️ @property Decorators y Descriptores

📄 ¿Qué son los Property Decorators?

Los property decorators son una forma elegante de implementar el patrón de descriptores en Python. Permiten definir atributos que se comportan como propiedades, con getters, setters y deleters controlados.

- ✓ @property: Convierte un método en atributo de solo lectura
- ✓ @nombre.setter: Define el comportamiento de asignación
- ✓ @nombre.deleter: Define el comportamiento de eliminación

💡 Relación con descriptores: @property crea un descriptor bajo el capó, implementando `__get__`, `__set__` y `__delete__`

🛡️ Ventajas de @property

Encapsulación

Control de acceso

Validación

Reglas de negocio

Caché

Optimización

> Ejemplo: Clase Cuenta con @property

```
class Cuenta:
    def __init__(self, saldo_inicial=0):
        self._saldo = saldo_inicial
    @property
    def saldo(self):
        """Getter para el saldo"""
        return self._saldo
    @saldo.setter
    def saldo(self, valor):
        """Setter con validación"""
        if valor < 0:
            raise ValueError("Saldo no puede ser negativo")
        self._saldo = valor
    @saldo.deleter
    def saldo(self):
        del self._saldo

# Uso:
cuenta = Cuenta(100)
print(cuenta.saldo)      # 100 (getter)
cuenta.saldo = 200      # Setter con validación
```

🔗 Descriptores vs @property

Descriptores son más flexibles pero requieren más código. @property es la forma "pythonica" de crear descriptores simples y legibles.

Participa: Introspección para Debugging



¿Cómo usar la introspección para depurar código en Python?

La introspección permite inspeccionar objetos en tiempo de ejecución. Usa `dir()`, `vars()`, `getattr()` y `setattr()` para diagnosticar problemas sin modificar el código fuente.



Guía de Discusión

- ✓ ¿Qué inspeccionar cuando algo falla? - `type`, `__dict__`, `vars`, `__class__`
- ✓ ¿Cómo automatizar con decoradores? - Crear decoradores de logging
- ✓ ¿Qué registrar sin filtrar datos sensibles? - Evitar passwords, tokens
- ✓ ¿Cómo identificar el tipo exacto? - `isinstance`, `type`, `__mro__`

Checklist de Depuración

- | | |
|--|---|
| ☒ Verificar tipo con <code>type()</code> | ☒ Inspeccionar atributos con <code>dir()</code> |
| ☐ Revisar <code>__dict__</code> del objeto | ☐ Verificar MRO con <code>__mro__</code> |
| ☐ Usar <code>getattr()</code> para valores | ☐ Validar con <code>isinstance()</code> |



Consejo Práctico

- 1 Crea un script de debugging reutilizable con funciones de introspección
- 2 Usa `pprint` para visualizar estructuras complejas de forma legible

 Trabajo en parejas

 8 min

 Iniciar

 Compartir

Casos de uso en frameworks de IA



PyTorch

nn.Module, autograd

nn.Module

- `__call__` para invocar forward
- ✓ Parámetros en `__dict__`
- ✓ `requires_grad` automático

```
class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(10, 5)
    def forward(self, x):
        return self.linear(x)
model = Model()
y = model(x)  # __call__ → forward
```



TensorFlow

Keras Layers

Layers

- `__call__` para invocar capas
- ✓ Atributos con property
- ✓ `build()` para crear pesos

```
class CustomLayer(Layer):
    def __init__(self, units):
        super().__init__()
        self.units = units
    def build(self, input_shape):
        self.w = self.add_weight(...)
    def call(self, inputs):
        return tf.matmul(inputs, self.w)
```



Datasets

DataLoader

`__len__` & `__getitem__`

- `__len__` para tamaño
- ✓ Indexado de datos
- ✓ Batching automático

```
class Dataset:
    def __len__(self):
        return len(self.data)
    def __getitem__(self, idx):
        return self.data[idx]
loader = DataLoader(dataset, batch_size=32)
```

Cuándo usar cada tipo de acceso



Público

Sin prefijo

✓ Cuándo usar

Atributos que forman parte de la API estable y documentada de la clase.

- ✓ API pública
- ✓ Documentación oficial
- ✓ Interfaz estable

```
class User:
    def __init__(self, name):
        self.name = name
        self.email = email
```



Protegido

Prefijo _

⚠ Cuándo usar

Atributos internos que pueden cambiar, pero señalan que no deben usarse fuera.

- ✓ Implementación interna
- ✓ Sujeto a cambios
- ✓ Herencia permitida

```
class User:
    def __init__(self):
        self._internal_id = 1
        self._cache = {}
```



Privado

Prefijo __

🔒 Cuándo usar

Atributos que deben evitar colisiones de nombres en herencia compleja.

- ✓ Evita colisiones
- ✓ Herencia compleja
- ✓ Name mangling

```
class User:
    def __init__(self):
        self.__password = ""
        self.__secret_key = ""
```



Dunder

Prefijo __

✎ Cuándo usar

Métodos especiales reservados para el modelo de datos de Python.

- ✓ Protocolos Python
- ✓ Métodos mágicos
- ✓ Integración con el lenguaje

```
class User:
    def __str__(self):
        return f"User({self.name})"
    def __repr__(self):
        return f"User(name={self.name})"
```

★ Mejores prácticas y patrones

✓ Patrones Recomendados

- ✓ Dataclasses: Usa @dataclass para reducir boilerplate en clases de datos
- ✓ Functools: Implementa total_ordering para comparaciones completas
- ✓ Contextlib: Usa @contextmanager para simplificar context managers
- ✓ Documentación: Documenta contratos y casos borde claramente

💡 Consejo: Implementa __repr__ útil para debugging y evita abusar de introspección en rutas críticas

✂ Mejores Prácticas



Implementa __repr__ útil
Para debugging efectivo



Usa dataclasses
Reduce código repetitivo



total_ordering
Para orden total



Documenta contratos
Casos borde incluidos

</> Ejemplo de Dataclass

```
from dataclasses import dataclass
@dataclass
class ModeloIA:
    nombre: str
    parametros: dict
    version: str = "1.0"
    def __repr__(self):
        return f"ModeloIA(nombre={self.nombre}, version={self.version})"
```

🛡 Context Managers

```
from contextlib import contextmanager
@contextmanager
def timer():
    import time
    start = time.time()
    yield
    print(f"Tiempo: {time.time()-start}s")
```

Reto Final: Diseña una clase "Pythonic"



Clase TensorWrapper

Implementa múltiples métodos mágicos y property para manejo de tensores

 8-10 min

 Nivel: Avanzado



Requisitos de implementación

Crea la clase TensorWrapper con los siguientes métodos mágicos:

- ✓ `__init__`: datos (lista/numpy), nombre
- ✓ `__repr__`, `__str__`: Representación informativa
- ✓ `__len__`, `__iter__`: Soporte para `len()` e iteración
- ✓ `__add__`, `__mul__`: Operaciones element-wise
- ✓ `__call__`: Aplicar función a elementos
- ✓ `property .shape`: Dimensión del tensor

```
class TensorWrapper:
    def __init__(self, datos, nombre=""):
        self.datos = datos
        self.nombre = nombre
```



Pistas y consejos

- ✓ Usa `numpy.array` para manejo de datos
- ✓ Implementa `__eq__` para comparación
- ✓ Valida tipos en operaciones
- ✓ Maneja errores con try-except

 **Tip:** Para `__call__`, permite aplicar funciones lambda o built-in a cada elemento.

 Ver solución

✓ Validar código



Resumen — Puntos clave



Convenciones

- ✓ `_` indica "protegido" por convención
- ✓ `__` activa name mangling
- ✓ Accesible vía `_Clase_.atributo`
- ✓ Comunica intención, no es seguridad

`_protegido`

`__privado`



Métodos mágicos

- ✓ `__init__`, `__str__`, `__repr__`
- ✓ `__eq__`, `__lt__`, `__gt__`
- ✓ `__add__`, `__sub__`, `__mul__`
- ✓ Integración con ecosistema Python



Protocolos

- ✓ Iteración: `__iter__`, `__next__`
- ✓ Contexto: `__enter__`, `__exit__`
- ✓ Invocación: `__call__`
- ✓ Mejora usabilidad y seguridad



Introspección

- ✓ `dir()`, `vars()`, `getattr()`
- ✓ `__dict__`, `__class__`
- ✓ Debugging y metaprogramación
- ✓ Inspección en tiempo de ejecución

Recuerda: Prioriza claridad, contratos y pruebas. Elige el nivel de acceso mínimo necesario para la intención.

 Descargar

 Compartir



Recursos y próximos pasos



Documentación y libros



Python Data Model

docs.python.org - Referencia oficial

[Ver documentación →](#)



Fluent Python

Luciano Ramalho - Mejores prácticas

[Ver en Amazon →](#)



PEP 8

Estilo de código Python

[Ver PEP →](#)



Ejercicios y práctica

- ✓ Refactoriza una clase con `__repr__`/`__eq__`
- ✓ Implementa un iterador personalizado
- ✓ Crea un context manager para recursos
- ✓ Diseña una clase con métodos mágicos



Próxima clase

Tema: Decoradores avanzados y metaclasses básicas

Objetivo: Dominar patrones de diseño avanzados



Recursos adicionales



Real Python

Tutoriales avanzados

[Ver tutoriales →](#)



GitHub

Ejemplos de código

[Ver repositorios →](#)



Comunidad

Stack Overflow

[Ver preguntas →](#)

Nota: Los recursos están seleccionados para reforzar los conceptos aprendidos en clase.



Descargar recursos



Compartir